of metrics is centralized. You can directly query those raw metrics in the Prometheus exposition format using the `kubectl get --raw` command, for debugging purposes. The following code provides an example (edited to fit):

Access the /metrics endpoint to get raw metrics, filter with grep for metrics that have apiserver in it, and limit with head to five lines.

```
$ kubectl get --raw /metrics | grep --color=auto apiserver | head -5
# HELP apiserver_admission_controller_admission_duration_seconds [ALPHA]
  Admission controller latency histogram in seconds, identified by name
  and broken out for each operation and API resource and type (validate
  or admit).
# TYPE apiserver_admission_controller_admission_duration_seconds histogram
apiserver_admission_controller_admission_duration_seconds_bucket{
name="CertificateApproval",operation="UPDATE",rejected="false",
type="validate",le="0.005"} 9.729562e+06
apiserver_admission_controller_admission_duration_seconds_bucket{
name="CertificateApproval",operation="UPDATE",rejected="false",
type="validate",le="0.025"} 9.729568e+06
apiserver_admission_controller_admission_duration_seconds_bucket{
name="CertificateApproval",operation="UPDATE",rejected="false",
type="validate",le="0.1"} 9.729568e+06
```

The metric shown here, apiserver_admission_controller_admission_duration_seconds, is a histogram type for a control plane component (the admission controller).

Traces are the most recently added signal type in Kubernetes. It uses OpenTelemetry (http://mng.bz/Zq6m) to capture and expose traces (at time of writing) of the API server.

3.2.4 Serverless compute

In this section, we take a look at signal sources in the context of serverless compute offerings. These serverless compute environments are usually preferable from an operational perspective compared to managing servers yourself; however, there are cases where you can't use them. Some of these situations include use cases with requirements to control certain low-level settings, such as Linux kernel modules or hardware configurations, for performance reasons. Another class of use cases comes from regulatory requirements, if certain certifications required for financial industry or health care industry workloads are not available.

If you can, however, make use of serverless compute for your workload, you should strongly consider it. Let's begin by discussing the general characteristics of serverless compute and implications of the environments on the observability aspects.

No (access to) servers

By definition, *serverless* means you don't deal with servers—that is, you do not provision or maintain servers. Naturally, these servers still exist in the accounts of your provider, but now, it's their responsibility to maintain them. One implication is that you typically don't have `ssh` access to a server, and hence, many of the statements related to `journalctl` in the previous section do not apply. What you, in contrast, are dealing